KV-Cache Scaling

Predicting the Context Limit Before You Hit It

Detailed explanation, requirement analysis, 2-week phase plan, deliverables, tools and implementation

Item	Detail
Domain	LLM memory management, GPU systems profiling
Original budget	Team of 4 x 2 h/week x 12 weeks = about 96 person-hours
Our plan	Team of 2, 2 weeks, about 12 h/week each = about 48 person-hours (about half the original budget, so scope is trimmed where noted)
Recommended model	Qwen2.5-1.5B (fp16), ungated and small; Llama-3.2-3B is the alternative
Hardware	Free Colab / Kaggle GPU (usually a 15 GB T4). Always record which GPU you got.

How to read this document. Sections 1-2 explain the problem. Section 3 is the requirement analysis. Section 4 is the technical design (tools and implementation). Section 5 is the 2-week plan for two people. Section 6 lists deliverables. Section 7 covers risks and questions to confirm with your instructor. Numbers marked *(verify)* are from my knowledge of the model configs; read them from the real config file yourselves, because the project explicitly requires that.

1. What this project is about

When an LLM generates text it stores, for every token it has already seen, two vectors (a key and a value) in every attention layer. This stored data is the **KV-cache**. It grows linearly with the number of tokens in the context, so a long enough context will exhaust GPU memory. The project asks you to treat this as an engineering prediction problem rather than a trial-and-error one:

- **Predict** on paper the longest sequence your GPU can hold, and commit that number before measuring anything.
- **Measure** real memory use as the sequence grows, carefully, without crashing the GPU over and over.
- **Explain** the difference between prediction and reality, megabyte by megabyte, naming each cause.
- **Generalise** the validated equation to models and GPUs you did not run.

The grade-worthy part is not the final number. It is the honest accounting: a prediction that was wrong, plus a quantitative explanation of why, with any leftover gap stated openly as *unaccounted*.

2. Concepts you must understand first

2.1 The memory equation

```
M(L,B) = M_weights + M_kv(L,B) + M_activations(L,B) + M_overhead

M_kv = 2 x B x L x n_layers x n_kv_heads x d_head x bytes_per_element
       ^ the 2 is one K tensor and one V tensor
```

Symbol	Meaning	Where to read it
L	Sequence length (tokens held in context)	You choose it
B	Batch size (sequences processed together)	You choose it (primary runs use B = 1)
n_layers	Number of transformer blocks	config.json: num_hidden_layers
n_kv_heads	Number of key/value heads (NOT the query head count)	config.json: num_key_value_heads
d_head	Size of each head	hidden_size / num_attention_heads, or head_dim if present
bytes_per_element	2 for fp16/bf16, 4 for fp32	Your chosen dtype

2.2 Grouped-query attention (GQA), the trap in the brief

In classic multi-head attention, every query head has its own key/value head, so n_kv_heads = n_heads. Modern models use GQA: many query heads share a few K/V heads. Qwen2.5-1.5B has 12 query heads but only 2 KV heads *(verify)*. If you use n_heads in the formula you overestimate the KV-cache by 6x for this model. The statement says catching this is part of the exercise, so show the correct and the naive calculation side by side in your report.

2.3 Worked example (Qwen2.5-1.5B, fp16) - recompute from the real config

Values *(verify)*: 28 layers, 2 KV heads, d_head = 1536 / 12 = 128.

```
KV bytes per token = 2 x 28 x 2 x 128 x 2 = 28,672 B = 28 KiB
Wrong (n_heads=12) = 2 x 28 x 12 x 128 x 2 = 172,032 B = 168 KiB   (6x too big)
```

L	512	1024	2048	4096	8192
KV-cache at B=1 (MiB)	14	28	56	112	224

Weights: about 1.54 B parameters x 2 bytes = about 3.1 GB (2.9 GiB) *(verify with sum(p.numel()))*. These tiny KV numbers lead to the most important design decision in this project, explained in section 3.3.

2.4 The three things that make real memory exceed the equation

- **Activations.** Temporary tensors during the forward pass: attention scores, MLP intermediates, and the final logits tensor (L x vocab_size). For Qwen the vocabulary is 151,936 tokens, so full logits at L = 8192 in fp16 is

about 2.5 GB, far more than the KV-cache at that length.

- **Allocator fragmentation.** PyTorch's caching allocator keeps freed blocks for reuse. memory_reserved() minus memory_allocated() is memory held but not in use. Fragmentation can make an allocation fail even though total free memory looks sufficient.
- **Framework overhead.** The CUDA context, cuBLAS/cuDNN workspaces and kernel modules take several hundred MB that PyTorch's counters do not show but nvidia-smi / pynvml does.

2.5 Three different memory counters

Counter	What it reports	Use it for
torch.cuda.memory_allocated()	Bytes currently occupied by live tensors	Weights + KV + live activations
torch.cuda.max_memory_allocated()	Peak of the above since last reset	Transient activation peak
torch.cuda.memory_reserved()	Bytes PyTorch obtained from the driver (allocated + cached)	Allocator gap = reserved - allocated
pynvml used / free / total	Whole-GPU view from the driver, includes CUDA context	Framework overhead = nvml used - reserved

3. Requirement analysis

3.1 Objective-by-objective breakdown

Each objective in the brief is turned into an explicit requirement, the hidden requirements it implies, and a pass/fail acceptance criterion.

#	Requirement (from brief)	Implied / hidden requirements	Acceptance criterion
1	Derive M(L,B) on paper with symbols defined; read real config values; handle GQA	Load config with AutoConfig, no GPU needed. Model activation terms too (logits, MLP, attention). Show naive vs GQA-correct KV.	Derivation document with every symbol defined, values traced to config.json, GQA comparison shown.
2	Pre-register prediction: max L at B=1, plus memory at L = 512, 1024, 2048, 4096, 8192	Must be committed BEFORE any sweep. Needs a timestamp that cannot be faked. Needs the GPU's usable memory (record GPU model).	Git commit (pushed) containing prediction file and derivation; commit hash and time quoted in report. No measurement code run for the sweep before that commit.
3	Sweep L upward, record allocated / reserved / free; bisection; stop before OOM; exactly ONE deliberate OOM; checkpoint after each point	Define precisely what one 'measurement' is. Safety rule to avoid accidental OOM. Resume logic. Count OOMs and log the one deliberate OOM.	Results file appended after every point; harness resumes after a runtime kill; exactly one OOM recorded, marked deliberate.
4	Separate weights, KV slope, residual; report residual fraction and explain it	Weights measured after load, before any forward pass. KV from regression slope. Residual = total - weights - KV.	Fitted slope within a few percent of 28,672 B/token (for Qwen); residual fraction table with explanation.
5	Quantify allocator gap; test expandable_segments:True; report delta in max usable context	Two separate processes (env var must be set before CUDA init). Same bisection for both. Same GPU.	Gap vs L plotted; max L with and without expandable_segments; difference stated in tokens and %.
6	Explain prediction error: attribute gap across activations, fragmentation, framework overhead; state unaccounted	Needs peak counters and nvml data from step 3 to build an accounting identity (section 4.5).	Discrepancy table whose rows sum to the total gap, with an explicit 'unaccounted' row.
7	Generalise: predict max context for 2 models and 2 GPUs not run; verify at least one against published figures	Read configs for the other models. Needs a defensible activation/overhead model carried over. Needs a published figure to compare.	Predictions table (2 models x 2 GPUs), one verified with percentage error against a cited source.

3.2 Functional and non-functional requirements

Type	Requirement
Functional	Load model in fp16; run a controlled forward pass at chosen (L, B); record counters; bisect to a boundary; plot measured vs analytical.
Functional	Persist every measurement to disk immediately (append-only JSONL plus GPU/model metadata).
Functional	Run the same experiment with and without PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True.
Non-functional	Reproducible: fixed seeds, pinned library versions, recorded GPU name, driver, torch version, attention implementation.
Non-functional	Crash-safe: a lost Colab runtime costs at most one data point. Results saved to Drive or pushed to Git.
Non-functional	Honest: no fitting the prediction after the fact; every gap either attributed or labelled unaccounted.
Constraint	Model 1B-3B in fp16. No 7B/8B model for the measured runs. Only one deliberate OOM.
Constraint	Compute limited to free-tier Colab/Kaggle sessions (time limits, GPU varies between sessions).

3.3 Critical design findings from the analysis

Finding 1 - single forward pass hides the KV-cache. For Qwen2.5-1.5B the KV-cache is only 28 KiB per token, so at L = 16,000 it is just 0.45 GiB. In a one-shot prefill (feeding all L tokens at once) memory is dominated by **activations**, mainly the logits tensor and attention scores, so the OOM point would be set by activations and the KV slope would be buried. **Recommendation:** implement two experiment modes. **Mode A (prefill):** one forward pass over L tokens. This measures activation behaviour and gives the 'realistic first-request' OOM. **Mode B (chunked prefill):** feed the context in chunks of 512 tokens while keeping past_key_values, so activations stay small and memory growth is almost pure KV. This isolates the slope that Objective 4 needs.

Finding 2 - model context limit vs GPU limit. Qwen2.5-1.5B is configured for about 32K positions *(verify max_position_embeddings)*. On a 15 GB T4 the KV-cache alone would not fill memory below that length (32K tokens = 0.9 GiB). Memory measurement does not need meaningful text, so use random token IDs and allow L beyond the configured limit, documenting that you did so, or sweep batch size B as well. Decide this on Day 1 and record it in the pre-registration. Llama-3.2-3B (112 KiB/token, weights about 6.4 GB) reaches its GPU limit sooner but is a gated download and leaves less headroom on a T4.

Finding 3 - define 'max sequence length' exactly. State in the pre-registration which mode, which attention implementation (sdpa vs eager; eager adds an L-squared attention-score term) and whether the full logits tensor is computed. Newer transformers versions accept logits_to_keep=1, which removes the largest activation. Changing these changes the answer by gigabytes, so freeze them before committing the prediction.

3.4 Assumptions and open questions

- The brief says team of 4 / 12 weeks; you are 2 people / 2 weeks. Confirm with your instructor that a trimmed scope (section 5) is acceptable.
- Confirm whether 'maximum sequence length' means a prefill of L tokens or a cache grown by decoding to length L. This plan supports both and pre-registers both.
- Confirm the instructor accepts Kaggle as well as Colab (Kaggle gives a more predictable GPU quota).

4. Technical design: tools and implementation

4.1 Tool stack

Purpose	Tool	Why
GPU environment	Google Colab free tier or Kaggle notebooks	Required by brief; T4 15 GB typical. Log nvidia-smi output at the start of every session.
Model + config	Hugging Face transformers, AutoConfig, AutoModelForCausalLM	Read real architecture values; run forward passes with use_cache=True.
Memory counters	torch.cuda.* memory APIs, pynvml (nvidia-ml-py)	Allocated / reserved / peak from PyTorch; total/used/free from driver.
Allocator experiment	PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True	Required intervention. Must be set before the process touches CUDA.
Data handling	pandas, numpy (np.polyfit), scipy.stats.linregress	Slope/intercept, R-squared, residual analysis.
Plots	matplotlib	Measured points with the analytical curve overlaid; allocator-gap plot.
Pre-registration	Git + GitHub (pushed commit) and optionally a signed tag	Server-side commit time is the tamper-evident timestamp.
Checkpointing	Append-only JSONL file on Drive/Kaggle output, one line per measurement	A crash loses at most the in-flight point.
Report	Markdown/LaTeX or Word, exported to PDF	Derivation with symbols, tables and figures.

4.2 Repository layout

```
kvcache-project/
  README.md                  how to run, GPU used, versions
  prereg/prediction.md       derivation + committed predictions (commit FIRST)
  src/model_math.py          closed-form M(L,B) from config; GQA vs naive
  src/probe.py               ONE measurement at (L,B,mode); prints a JSON line
  src/sweep.py               driver: schedule, bisection, safety rule, resume
  src/analysis.py            regression, residual split, gap table
  src/generalize.py          other models / GPUs predictions
  results/sweep_default.jsonl, results/sweep_expandable.jsonl
  figures/                   PNG plots
  report/                    final PDF
```

4.3 Analytical model (src/model_math.py)

```
from transformers import AutoConfig
cfg = AutoConfig.from_pretrained('Qwen/Qwen2.5-1.5B')
n_layers = cfg.num_hidden_layers
n_kv     = cfg.num_key_value_heads            # GQA-aware
d_head   = getattr(cfg, 'head_dim', None) or cfg.hidden_size // cfg.num_attention_heads
kv_per_token = 2 * n_layers * n_kv * d_head * 2   # bytes, fp16

def m_kv(L, B=1):  return B * L * kv_per_token
def m_logits(L, B=1, logits_dtype_bytes=2):        # prefill, full logits
    return B * L * cfg.vocab_size * logits_dtype_bytes
def m_total(L, B=1, W=None, overhead=None, act=None): ...
# activations: start with logits + MLP intermediate (L*intermediate_size*2 B)
# + attention (sdpa ~ O(L), eager adds B*n_heads*L*L*2 B). Keep the terms explicit.
```

Activation and overhead terms in the *pre-registered* prediction must come from reasoning (tensor shapes, typical CUDA context size), not from a trial run. After measurement you will replace them with fitted values for the generalisation step and show both.

4.4 The measurement probe and sweep harness

Run **each measurement in a fresh subprocess**. This gives a clean allocator for every point, protects the notebook from a crash, and makes expandable_segments comparison fair. The cost is reloading the model (about 10-20 s once cached on disk).

```
# src/probe.py  (one measurement)  usage: python probe.py --L 4096 --B 1 --mode prefill
import torch, json, pynvml, time, sys
def snap():
    pynvml.nvmlInit(); h = pynvml.nvmlDeviceGetHandleByIndex(0)
    i = pynvml.nvmlDeviceGetMemoryInfo(h)
    return dict(alloc=torch.cuda.memory_allocated(), peak=torch.cuda.max_memory_allocated(),
                reserved=torch.cuda.memory_reserved(), nvml_used=i.used,
                nvml_free=i.free, nvml_total=i.total)

model = load_fp16_model()                       # AutoModelForCausalLM, torch_dtype=float16
torch.cuda.synchronize(); w = snap()            # weights-only snapshot, no forward pass yet
torch.cuda.reset_peak_memory_stats()
ids = torch.randint(0, cfg.vocab_size, (B, L), device='cuda')
with torch.no_grad():
    out = run_mode(model, ids, mode)            # prefill, or chunked 512 with past_key_values
torch.cuda.synchronize(); after = snap()        # cache still alive here
print(json.dumps(dict(L=L, B=B, mode=mode, weights=w, after=after,
                      gpu=torch.cuda.get_device_name(0), t=time.time())))

# src/sweep.py  (driver, runs probe.py via subprocess, appends to JSONL, resumes)
def done_points(path): ...                      # read JSONL, return set of (L,B,mode,env)
def run_point(L,B,mode,env): ...                # subprocess.run, parse last JSON line, append+flush+fsync

# Phase 1: geometric ladder 256,512,1024,...  Stop climbing when predicted_peak(L) > 0.92 * nvml_total
# Phase 2: fit peak(L) on measured points (quadratic if eager, else linear), extrapolate
# Phase 3: bisection on the *predicted* boundary L*, only ever testing L whose extrapolated
#          peak is below a safety margin, so no accidental OOM; tighten margin as fit improves
# Phase 4: ONE deliberate OOM at L*(1+delta); catch torch.cuda.OutOfMemoryError in the probe,
#          write {oom: true, deliberate: true} to JSONL, and never repeat
```

Safety rule: a point is only attempted if the extrapolated peak memory from earlier points stays below a margin (start at 92% of total, tighten to about 97%). The deliberate OOM is attempted once, at a value just above the best bisection estimate, to confirm the boundary. Keep an OOM counter in the JSONL so the 'exactly one' rule is auditable.

4.5 Analysis: separating components and building the discrepancy table

Use this accounting identity, which makes the gap attributable rather than hand-waved:

```
nvml_used = CUDA_context_and_other  +  reserved
reserved  = allocated_peak  +  allocator_gap   (cached, unused blocks)
allocated_peak = weights + KV(L) + activation_peak(L)
```

- **Weights:** allocated measured right after load (before forward). Compare to params x 2 B.
- **KV slope:** linear regression of post-forward allocated (cache still alive, activations freed) against L. Slope should match kv_per_token; intercept = weights + small residual. Report R-squared and % error of slope.
- **Activation peak:** max_memory_allocated minus post-forward allocated. Compare to your tensor-shape prediction (logits, MLP, attention).
- **Allocator gap:** reserved - allocated at each L; report maximum, and how it changes with expandable_segments.
- **Framework overhead:** nvml_used - reserved (CUDA context, cuBLAS workspace).
- **Unaccounted:** total predicted-vs-measured gap minus the sum of the three attributed terms. Report it plainly.

Template for the discrepancy table (fill with measured values):

Component	Predicted (MB)	Measured (MB)	Delta (MB)	Explanation
Weights				e.g. tied embeddings, buffers
KV-cache at L*				slope error, cache pre-allocation
Activations (peak)				logits, MLP, attention workspace
CUDA context / overhead				nvml_used - reserved
Allocator fragmentation				reserved - allocated
Unaccounted				stated explicitly
TOTAL gap (L* predicted vs measured)				rows must sum to this

4.6 Generalisation (src/generalize.py)

```
L_max = (VRAM_usable - W - context_overhead - activation_fit(L)) / kv_per_token
VRAM_usable = nominal VRAM x (nvml_total / nominal ratio seen on your GPU)
```

- **Models you do not run (examples):** Llama-3.2-3B, Qwen2.5-7B, Mistral-7B or Llama-3.1-8B. Read their configs (AutoConfig works without downloading weights) and compute kv_per_token and W. Example *(verify)*: Llama-3.1-8B with 32 layers, 8 KV heads, d_head 128 gives 128 KiB per token.
- **GPUs you do not have:** A100-40GB, RTX 4090 (24 GB), L4 (24 GB), A10G (24 GB). Use their usable-memory fraction and your fitted overhead and activation coefficients.
- **Verification:** compare one prediction with a published figure. A practical source is the 'GPU KV cache size: N tokens' line that vLLM prints at start-up for a given model, GPU and memory-utilisation setting, or a published benchmark/blog table. Cite the source and report the percentage error. State differences in assumptions (e.g. vLLM reserves a utilisation fraction and uses paged blocks).

5. Two-person, two-week phase plan

Roles: **Member A** = theory and analysis lead (derivation, prediction, analysis, generalisation). **Member B** = engineering lead (harness, runs, plots). Both review each other's work and both attend the pre-registration commit. Assumed effort is about 12 h per person per week; Colab sessions are free-tier so book GPU time early and run long jobs back-to-back.

Phase	Days	Goal	Member A	Member B	Exit criterion
0 Setup and decisions	1	Environment and scope frozen	Read config, finalise model choice, define 'max L' (mode, attention, logits), draft report skeleton	Create repo, Colab/Kaggle notebook, install deps, confirm GPU, test pynvml; no sweep yet	Decisions written in README; GPU recorded
1 Derivation and pre-registration	2-3	Prediction committed	Full derivation, GQA comparison, predictions at 512/1K/2K/4K/8K and max L; write prereg/prediction.md	Write model_math.py and unit-test against hand numbers; write probe.py skeleton (do not run sweep)	Pushed git commit with prediction. Hash and time saved. THIS GATES ALL MEASUREMENT.
2 Harness	4-6	Crash-safe probe and sweep	Define safety margin, decide ladder values, write analysis.py draft on dummy data	Finish probe.py, sweep.py (JSONL, resume, subprocess per point, bisection, OOM handling); dry-run at small L	Harness resumes after a forced kill; points appended correctly
3 Measurement	7-9	Real data collected	Monitor sweeps, record GPU info, sanity-check numbers, start report sections 1-3	Run Mode B and Mode A sweeps; weights snapshot; bisection; one deliberate OOM; then expandable_segments runs	Both JSONL files complete; one OOM logged; max L found for both settings
4 Analysis	10-11	Gap explained	Regression for KV slope, residual fraction, discrepancy table, unaccounted term	Plots: measured vs analytical overlay, allocator gap vs L, expandable_segments comparison	Table rows sum to total gap; plots generated from saved data
5 Generalise	12	Cross-model / GPU predictions	Compute 2 models x 2 GPUs; find published figure; compute error	Add generalize.py and prediction plot; reproduce numbers from clean run of repo	Verification error reported with source
6 Finalise	13-14	Deliverables packaged	Write results, discussion, limitations; proofread	Clean README, final figures, export PDF, tag release; buffer for re-runs	All deliverables checklist (section 6) ticked

5.1 What to trim from the 12-week design

- Run the primary sweep for B = 1 only. Sweep B only if the model context limit prevents reaching an OOM (Finding 2).
- Use coarse ladders (powers of two) plus bisection near the end; do not sweep every L.
- Run expandable_segments as one extra sweep, not a full repeat of all modes (run Mode A at least).
- Generalise to exactly two models and two GPUs; verify exactly one against a published number.
- Keep a half-day buffer in Phase 6 for a failed Colab session or a re-run.

5.2 Week summary

Week	Outcome
Week 1 (Days 1-7)	Scope frozen, derivation and prediction committed, harness written and tested, first sweep points collected.
Week 2 (Days 8-14)	Sweeps and OOM boundary finished, expandable_segments comparison, analysis and plots, generalisation, final report.

6. Deliverables

Deliverable (from brief)	What we will produce	Owner	Done when
Derivation with symbols defined	Report section plus prereg/prediction.md: M(L,B), GQA-correct KV, activation terms, worked numbers	A	Every symbol defined; config values cited
Pre-registered prediction with commit timestamp	Predicted max L at B=1 and memory at 512/1K/2K/4K/8K; commit hash and time	A (B reviews)	Commit exists before first sweep
Checkpointing sweep harness	probe.py, sweep.py with JSONL, resume, bisection, safety rule, one deliberate OOM	B	Survives a forced kill; documented usage
Memory vs L plots with analytical overlay	Measured allocated/peak/reserved points with the analytical curve; allocator-gap plot; expandable_segments comparison	B	Figures regenerate from saved data
Discrepancy table	Component-by-component table with 'unaccounted' row	A	Rows sum to total gap
Cross-model / cross-GPU predictions	2 models x 2 GPUs table; one verified with % error and cited source	A + B	Source cited, error stated
Final report	PDF with results, discussion, limitations, reproducibility info (GPU, versions)	Both	Proofread and exported
Repository	Clean repo, README, results data, figures	B	Another person can rerun

6.1 Final report outline

- 1. Setup: model, GPU actually allocated, library versions, experiment definition (mode, attention, logits).
- 2. Derivation and pre-registered prediction (with commit hash and time).
- 3. Method: harness design, bisection, safety rule, deliberate OOM.
- 4. Results: curves with analytical overlay; KV slope validation; residual fraction.
- 5. Allocator gap and expandable_segments effect.
- 6. Prediction error attribution (discrepancy table).
- 7. Generalisation and verification.
- 8. Limitations and what remains unaccounted.

7. Risks and mitigations

Risk	Impact	Mitigation
Colab disconnects or gives a different GPU	Lost data or non-comparable numbers	Append-only JSONL after every point; GPU name stored in every record; run comparisons within one session
Accidental OOM before the deliberate one	Violates 'one OOM' rule	Extrapolation-based safety margin; subprocess isolation; OOM counter in log
Prediction made after seeing data	Pre-registration invalid	Push the commit before the first sweep; both members confirm; do not edit the file afterwards (add a dated addendum instead)
KV-cache buried by activations	Slope cannot be measured	Use Mode B (chunked prefill) and logits_to_keep=1 as a controlled variant
Model context limit stops sweep before OOM	Cannot find the boundary at B=1	Use random token IDs beyond the configured limit (documented) or sweep B
expandable_segments env var set too late	Invalid comparison	Set in subprocess environment before import torch
Library version differences (logits handling, attention backend)	Predictions off by GBs	Pin versions; record attn_implementation; freeze definition in pre-registration
Time overrun	Missed deliverables	Phase 6 buffer; trim list in 5.1

7.1 Definition of done

- Prediction committed and timestamped before any sweep.
- Sweep data for default and expandable_segments settings saved; exactly one deliberate OOM logged.
- KV slope validated against the analytical value; residual fraction reported and explained.
- Discrepancy table complete with an explicit unaccounted row.
- Two models and two GPUs predicted, at least one verified against a published figure with error stated.
- Report and repository reproducible by someone else.